

8086 Assembler directives & Interfacing memory and I/O ports

Sk. Shalauddin Kabir

Lecturer,

Dept. of Computer Science and Engineering,
Jashore University of Science and Technology.

- **Instructions to the Assembler regarding the program being executed.**
- **Control the generation of machine codes and organization of the program; but no machine codes are generated for assembler directives.**
- **Also called 'pseudo instructions'**
- **Used to :**
 - **specify the start and end of a program**
 - **attach value to variables**
 - **allocate storage locations to input/ output data**
 - **define start and end of segments, procedures, macros etc..**

DB**DW****SEGMENT
ENDS****ASSUME****ORG
END
EVEN
EQU****PROC
FAR
NEAR
ENDP****SHORT****MACRO
ENDM**

- Define Byte
- Define a byte type (8-bit) variable
- Reserves specific amount of memory locations to each variable
- Range : $00_H - FF_H$ for unsigned value; $00_H - 7F_H$ for positive value and $80_H - FF_H$ for negative value
- General form : **variable DB value/ values**

Example:

```
LIST DB 7FH, 42H, 35H
```

Three consecutive memory locations are reserved for the variable LIST and each data specified in the instruction are stored as initial value in the reserved memory location

Assemble Directives

DB

DW

SEGMENT
ENDS

ASSUME

ORG
END
EVEN
EQU

PROC
FAR
NEAR
ENDP

SHORT

MACRO
ENDM

- Define Word
- Define a word type (16-bit) variable
- Reserves two consecutive memory locations to each variable
- Range : $0000_H - FFFF_H$ for unsigned value;
 $0000_H - 7FFF_H$ for positive value and
 $8000_H - FFFF_H$ for negative value
- General form : **variable DW value/ values**

Example:

ALIST DW 6512H, 0F251H, 0CDE2H

Six consecutive memory locations are reserved for the variable ALIST and each 16-bit data specified in the instruction is stored in two consecutive memory location.

Assemble Directives

DB

DW

**SEGMENT
ENDS**

ASSUME

**ORG
END
EVEN
EQU**

**PROC
FAR
NEAR
ENDP**

SHORT

**MACRO
ENDM**

- **SEGMENT** : Used to indicate the beginning of a code/ data/ stack segment
- **ENDS** : Used to indicate the end of a code/ data/ stack segment
- **General form:**

Segnam SEGMENT

...
...
...
...
...
...

Segnam ENDS

**Program code
or
Data Defining Statements**

**User defined name of
the segment**

Assemble Directives

DB

DW

**SEGMENT
ENDS**

ASSUME

**ORG
END
EVEN
EQU**

**PROC
FAR
NEAR
ENDP**

SHORT

**MACRO
ENDM**

- Informs the assembler the name of the program/ data segment that should be used for a specific segment.

- General form:

ASSUME segreg : segnam, .. , segreg : segnam

Segment Register

User defined name of
the segment

Example:

ASSUME CS: ACODE, DS:ADATA

Tells the compiler that the instructions of the program are stored in the segment ACODE and data are stored in the segment ADATA

Assemble Directives

DB

DW

**SEGMENT
ENDS**

ASSUME

**ORG
END
EVEN
EQU**

**PROC
FAR
NEAR
ENDP**

SHORT

**MACRO
ENDM**

- **ORG** (Origin) is used to assign the starting address (Effective address) for a program/ data segment
- **END** is used to terminate a program; statements after END will be ignored
- **EVEN** : Informs the assembler to store program/ data segment starting from an even address
- **EQU** (Equate) is used to attach a value to a variable

Examples:

ORG 1000H	Informs the assembler that the statements following ORG 1000H should be stored in memory starting with effective address 1000 _H
LOOP EQU 10FEH	Value of variable LOOP is 10FE _H
<pre> _SDATA SEGMENT ORG 1200H A DB 4CH EVEN B DW 1052H _SDATA ENDS </pre>	In this data segment, effective address of memory location assigned to A will be 1200 _H and that of B will be 1202 _H and 1203 _H .

Assemble Directives

DB

DW

**SEGMENT
ENDS**

ASSUME

**ORG
END
EVEN
EQU**

**PROC
ENDP
FAR
NEAR**

SHORT

**MACRO
ENDM**

- **PROC** Indicates the beginning of a procedure
- **ENDP** End of procedure
- **FAR** Intersegment call
- **NEAR** Intrasegment call
- **General form**

procname PROC[NEAR/ FAR]

...
...
...

RET

procname ENDP

} Program statements of the
procedure

} Last statement of the
procedure

User defined name of
the procedure

DB**DW****SEGMENT
ENDS****ASSUME****ORG
END
EVEN
EQU****PROC
ENDP
FAR
NEAR****SHORT****MACRO
ENDM**

Examples:

ADD64 PROC NEAR...
...
...**RET
ADD64 ENDP**

The subroutine/ procedure named **ADD64** is declared as **NEAR** and so the assembler will code the **CALL** and **RET** instructions involved in this procedure as near call and return

CONVERT PROC FAR...
...
...**RET
CONVERT ENDP**

The subroutine/ procedure named **CONVERT** is declared as **FAR** and so the assembler will code the **CALL** and **RET** instructions involved in this procedure as far call and return

Assemble Directives

DB

DW

**SEGMENT
ENDS**

ASSUME

**ORG
END
EVEN
EQU**

**PROC
ENDP
FAR
NEAR**

SHORT

**MACRO
ENDM**

- Reserves one memory location for 8-bit signed displacement in jump instructions

Example:

**JMP SHORT
AHEAD**

The directive will reserve one memory location for 8-bit displacement named AHEAD

Assemble Directives

DB

DW

**SEGMENT
ENDS**

ASSUME

**ORG
END
EVEN
EQU**

**PROC
ENDP
FAR
NEAR**

SHORT

**MACRO
ENDM**

- **MACRO** Indicate the beginning of a macro
- **ENDM** End of a macro
- **General form:**

macroname MACRO[Arg1, Arg2 ...]

...
...
...

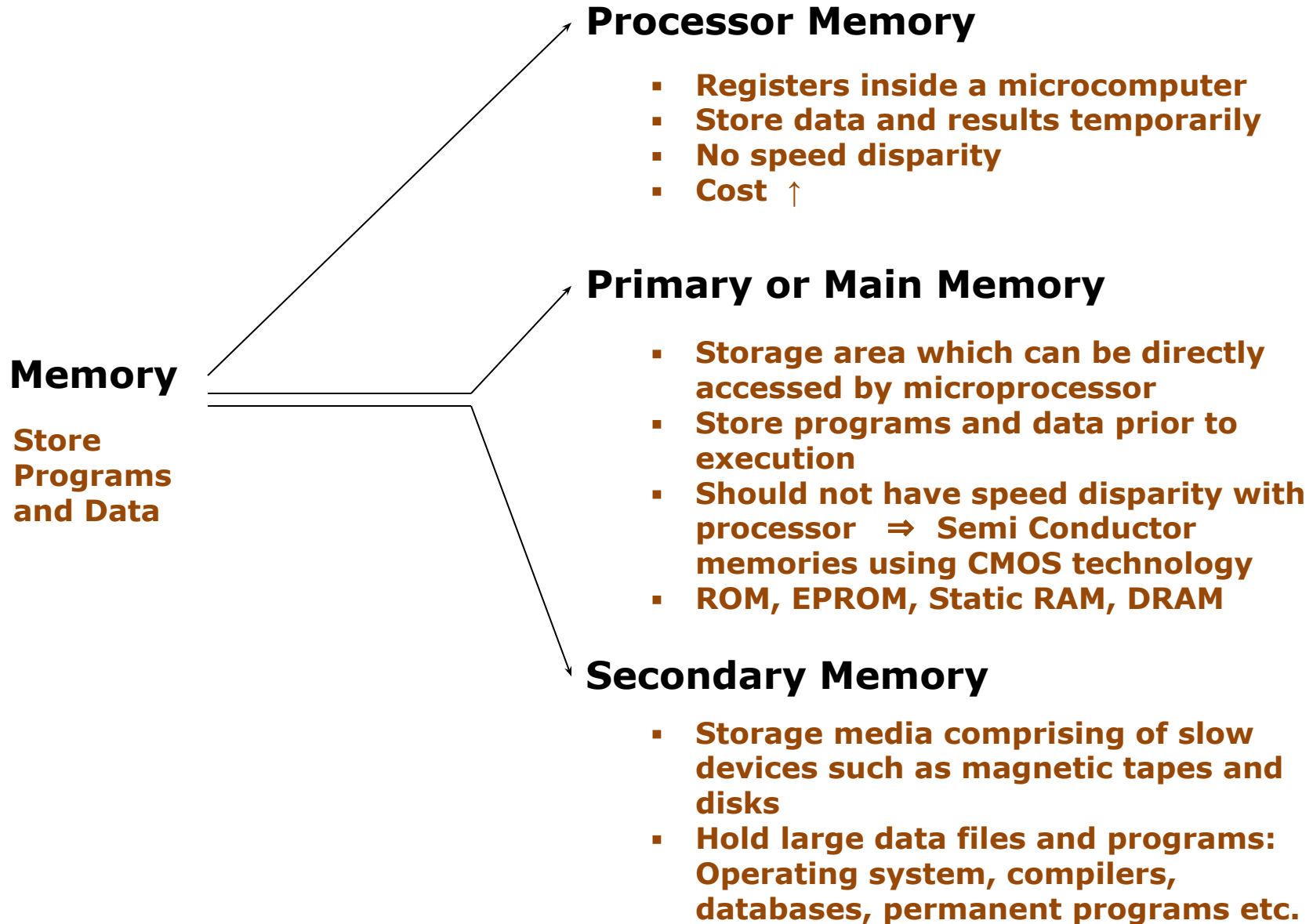


**Program
statements in
the macro**

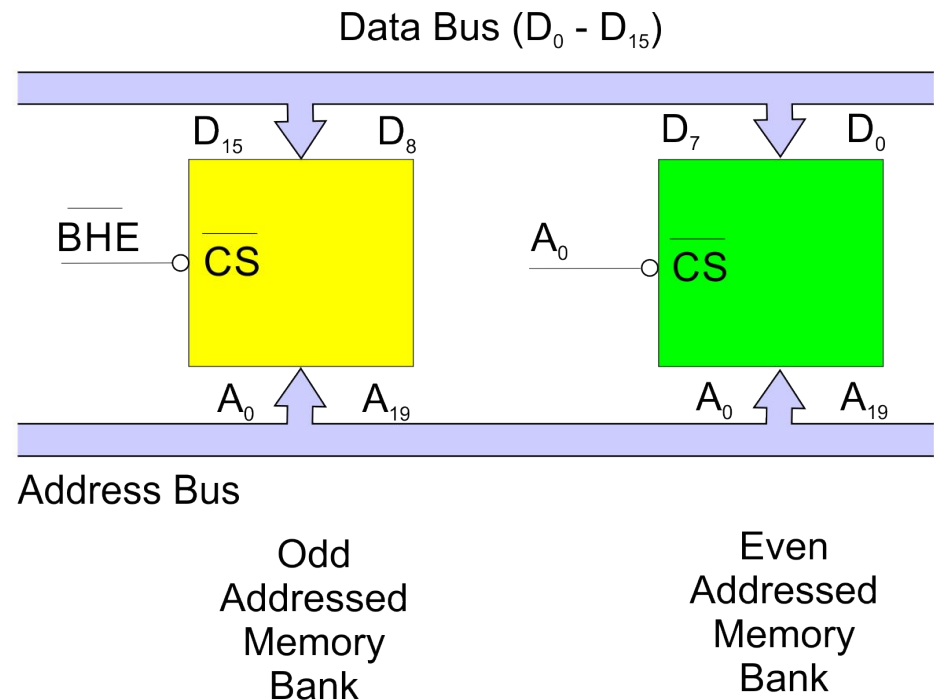
macroname ENDM

**User defined name of
the macro**

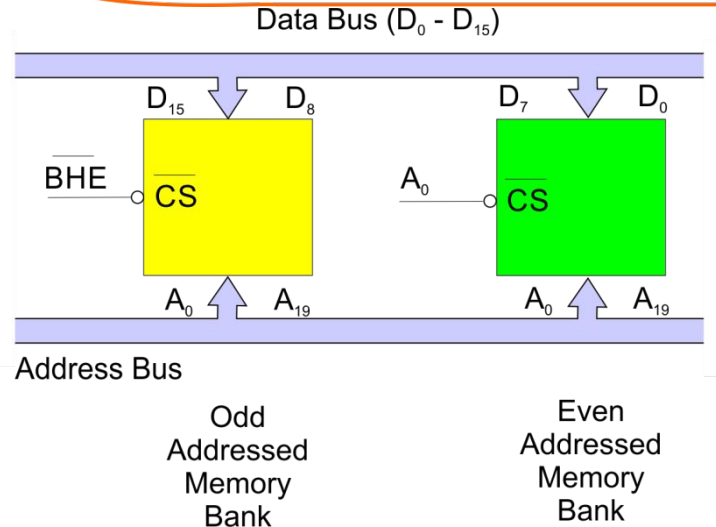
Interfacing memory and I/O ports



- **Memory IC's : Byte oriented**
- **8086 : 16-bit**
- **Word : Stored by two consecutive memory locations; for LSB and MSB**
- **Address of word : Address of LSB**
- **Bank 0 : $A_0 = 0 \Rightarrow$ Even addressed memory bank**
- **Bank 1 : $\overline{BHE} = 0 \Rightarrow$ Odd addressed memory bank**



Memory organization in 8086



	Operation		A ₀	Data Lines Used
1	Read/ Write byte at an even address	1	0	D ₇ - D ₀
2	Read/ Write byte at an odd address	0	1	D ₁₅ - D ₈
3	Read/ Write word at an even address	0	0	D ₁₅ - D ₀
4	Read/ Write word at an odd address	0	1	D ₁₅ - D ₀ in first operation byte from odd bank is transferred
		1	0	D ₇ - D ₀ in first operation byte from odd bank is transferred

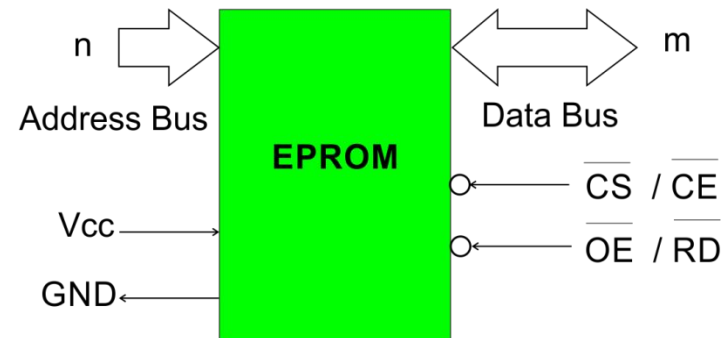
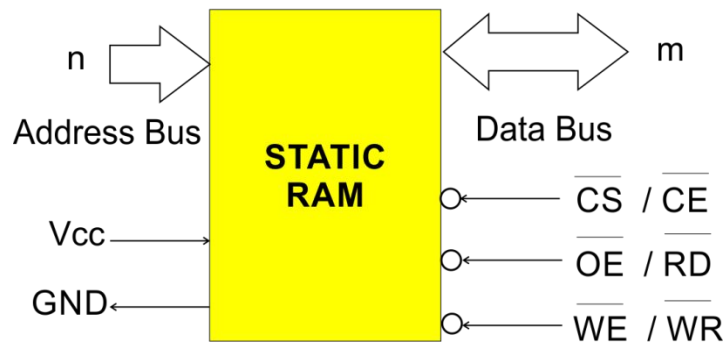
- Available memory space = EPROM + RAM
- Allot equal address space in odd and even bank for both EPROM and RAM
- Can be implemented in two IC's (one for even and other for odd) or in multiple IC's

- **Memory interface $\Rightarrow$ Read from and write in to a set of semiconductor memory IC chip**
- **EPROM $\Rightarrow$ Read operations**
- **RAM $\Rightarrow$ Read and Write**

In order to perform read/ write operations,

- **Memory access time $<$ read / write time of the processor**
- **Chip Select (CS) signal has to be generated**
- **Control signals for read / write operations**
- **Allot address for each memory location**

- Typical Semiconductor IC Chip



No of Address pins	Memory capacity			Range of address in hexa
	In Decimal	In kilo	In hexa	
20	$2^{20} = 10,48,576$	1024 k = 1M	100000	00000 to FFFFF

Monitor Programs

⇒ Programing 8279 for keyboard scanning and display refreshing

⇒ Programming peripheral IC's 8259, 8257, 8255, 8251, 8254 etc

⇒ Initialization of stack

⇒ Display a message on display (output)

⇒ Initializing interrupt vector table

Note :

8279	Programmable keyboard/ display controller
8257	DMA controller
8259	Programmable interrupt controller
8255	Programmable peripheral interface

I/O devices

⇒ For communication between microprocessor and outside world

⇒ Keyboards, CRT displays, Printers, Compact Discs etc.

⇒



⇒ Data transfer types

Programmed I/ O

Data transfer is accomplished through an I/O port controlled by software

Interrupt driven I/ O

I/O device interrupts the processor and initiate data transfer

Direct memory access

Data transfer is achieved by bypassing the microprocessor

Memory mapped

I/O mapped

Memory mapping	I/O mapping
20 bit address are provided for I/O devices The I/O ports or peripherals can be treated like memory locations and so all instructions related to memory can be used for data transmission between I/O device and processor	8-bit or 16-bit addresses are provided for I/O devices Only IN and OUT instructions can be used for data transfer between I/O device and processor
Data can be moved from any register to ports and vice versa	Data transfer takes place only between accumulator and ports
When memory mapping is used for I/O devices, full memory address space cannot be used for addressing memory. ⇒ Useful only for small systems where memory requirement is less	Full memory space can be used for addressing memory. ⇒ Suitable for systems which require large memory capacity